

Appendix A - Python Code

March 20, 2026

This appendix contains all Python code used for data processing, analysis, and visualization in support of the research article “Health Care Utilization and Immigration Enforcement.”

1 Environment Setup

Import required libraries.

```
[16]: # Import libraries
import pandas as pd
import numpy as np
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
from sodapy import Socrata
import janitor
```

2 Data Loading

Load the datasets for analysis.

2.1 CDC PLACES: Routine checkup within the past year among adults (outcome, 2018-2023)

This data is available from CDC’s PLACES data and provides information on the percentage of adults who reported having a routine checkup within the past year, broken down by county, zip, census tract, and year from 2020-2025 (data years from 2018-2023). This dataset will serve as our primary outcome variable, allowing us to assess health care utilization patterns in relation to local enforcement intensity.

Load PLACES: Local Data for Better Health:

- 2025 releases (2023 data):
 - [Census Tract Data](#)
 - [County Data](#)

- [ZCTA Data](#)
- 2024 releases (2022 data):
 - [Census Tract Data](#)
 - [County Data](#)
 - [ZCTA Data](#)
- 2023 releases (2021 data):
 - [Census Tract Data](#)
 - [County Data](#)
 - [ZCTA Data](#)
- 2022 releases (2020 data):
 - [Census Tract Data](#)
 - [County Data](#)
 - [ZCTA Data](#)
- 2021 releases (2019 data):
 - [Census Tract Data](#)
 - [County Data](#)
 - [ZCTA Data](#)
- 2020 releases (2018 data):
 - [Census Tract Data](#)
 - [County Data](#)
 - [ZCTA Data](#)

Note: Data released each year reflects BRFSS data from 2 years prior. For e.g., 2025 release reflects 2023 BRFSS data. 2016-2019 release data is also available, but data format is not consistent.

Richie, there is documentation on how to load this data via API [here](#), but I couldn't figure it out without hitting the rate limit! Downloading it manually for now, but let me know if this is something you want to try. - Naomi

```
[3]: # Load CDC PLACES data
# Data source: CDC PLACES
# Time period: 2018-2023
# Geographic levels: County, ZIP code, Census tract

file_templates = {
    "census": "Data/
↳PLACES__Local_Data_for_Better_Health,_Census_Tract_Data_{release}_release.
↳csv",
    "county": "Data/
↳PLACES__Local_Data_for_Better_Health,_County_Data_{release}_release.csv",
    "zip": "Data/
↳PLACES__Local_Data_for_Better_Health,_ZCTA_Data_{release}_release.csv",
}

for release_year in range(2025, 2019, -1): # 2025, 2024, ..., 2020
    data_year = release_year - 2
```

```

globals()[f"places_census_{data_year}"] = pd.read_csv(
    file_templates["census"].format(release=release_year)
)
globals()[f"places_county_{data_year}"] = pd.read_csv(
    file_templates["county"].format(release=release_year)
)
globals()[f"places_zip_{data_year}"] = pd.read_csv(
    file_templates["zip"].format(release=release_year)
)

```

2.2 TRAC Detention Facilities Data (2019-2026)

Load average daily population data from ICE detention facilities. This data is at the zip level.

Richie, I pulled this data manually from <https://tracreports.org/immigration/detentionstats/facilities.html> by copying and pasting it into an Excel file, but we could try scraping it instead if we want?

```

[7]: # Load TRAC detention facilities data
# Data source: TRAC (Transactional Records Access Clearinghouse), retrieved 3/
↳19/2026
# Time period: September 30, 2019 - February 5, 2026

detention_facilities = pd.read_excel('Data/
↳TRAC_DetentionFacilitiesAverageDailyPopulation.xlsx', skiprows=1)

```

2.3 TRAC ICE Detention Data (2015-2019)

Load monthly counts of individuals detained by ICE at the county level with demographic breakdowns.

Richie, any ideas on how to pull in the data here: <https://tracreports.org/phptools/immigration/detention/>? Unfortunately since it ends in 2019, it only overlaps with two years of our outcome data, but it could be interesting to get citizenship, gender, etc. by county.

```

[ ]: # Load TRAC ICE detention data (county level)
# Data source: TRAC
# Time period: March 2015 - July 2019

ice_detention = spark.read.csv('path/to/ice_detention.csv', header=True,
↳inferSchema=True)

# Placeholder for demonstration
print("Load TRAC ICE Detention data here")
# ice_detention.printSchema()
# ice_detention.show(5)

```

3 Data Cleaning and Preprocessing

Clean, standardize, and prepare datasets for analysis.

3.1 PLACES

Focusing on the data by zip code, we append data across years, handle missings, and fix data types.

```
[60]: # Data cleaning steps:
# 1. Handle missing values
# 2. Standardize geographic identifiers (county FIPS codes, ZIP codes)
# 3. Convert date formats
# 4. Remove duplicates
# 5. Validate data quality

# Append places_zip_2018 - 2023
places_zip_all_years = pd.concat(
    [globals()[f"places_zip_{year}"] for year in range(2018, 2024)],
    ignore_index=True
)

# Standardize column names and remove empty columns
places_zip_all_years = places_zip_all_years.clean_names().remove_empty()

# ZIP codes should be zero-padded 5-digit strings
places_zip_all_years['zip'] = (
    places_zip_all_years['locationname']
    .astype(int)
    .astype(str)
    .str.zfill(5)
)

# Fix column data types
places_zip_all_years['totalpopulation'] = pd.to_numeric(
    places_zip_all_years['totalpopulation']
    .astype(str)
    .str.replace(',', '', regex=False)
    .str.strip(),
    errors='coerce'
)

# Drop the original locationname and locationid columns since we have the
↳ standardized zip code
places_zip_all_years = places_zip_all_years.drop(columns=['locationname'])
places_zip_all_years = places_zip_all_years.drop(columns=['locationid'])

# Rename data_value to more descriptive name
```

```

places_zip_all_years = places_zip_all_years.rename(columns={'data_value':
↳ 'doctor_visits_past_year'})

# Drop measure name
places_zip_all_years = places_zip_all_years.drop(columns=['measure'])

# Since totalpop18plus has significant missing values, we will drop it for now
places_zip_all_years = places_zip_all_years.drop(columns=['totalpop18plus'])

# Remove columns that have the same value for all rows--these are not useful
↳ for modeling
for col in places_zip_all_years.columns:
    if places_zip_all_years[col].nunique() == 1:
        places_zip_all_years = places_zip_all_years.drop(columns=[col])

# Drop duplicates, if any
places_zip_all_years = places_zip_all_years.drop_duplicates()

places_zip_all_years.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 185504 entries, 0 to 185503
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   year                                  185504 non-null  int64
1   doctor_visits_past_year              185504 non-null  float64
2   low_confidence_limit                 185504 non-null  float64
3   high_confidence_limit                 185504 non-null  float64
4   totalpopulation                       185504 non-null  int64
5   geolocation                           185504 non-null  object
6   zip                                   185504 non-null  object
dtypes: float64(3), int64(2), object(2)
memory usage: 9.9+ MB

```

```
[53]: places_zip_all_years.head()
```

```

[53]:   year  doctor_visits_past_year  low_confidence_limit  high_confidence_limit \
0  2018                        80.5                    80.2                    80.9
1  2018                        75.6                    75.1                    76.1
2  2018                        70.5                    68.3                    72.5
3  2018                        78.1                    77.4                    78.8
4  2018                        77.5                    76.9                    78.0

   totalpopulation  geolocation  zip
0          16,769  POINT (-72.62581515 42.06255509)  01001
1          29,049  POINT (-72.4509085 42.38758794)  01002

```

2	10,372	POINT (-72.52475918 42.39190728)	01003
3	5,079	POINT (-72.10611705 42.42009493)	01005
4	14,649	POINT (-72.40031647 42.27869142)	01007

3.2 TRAC

We standardize columns names, correct data types, and handle missings.

```
[56]: detention_facilities_clean = detention_facilities.copy()

# Use janitor to clean column names
detention_facilities_clean = detention_facilities_clean.
    ↪ clean_names(remove_special=True)

# Drop if zip code is missing (not useful for analysis)
detention_facilities_clean = detention_facilities_clean.dropna(subset=['zip'])

# Update data types. ZIP codes should be zero-padded 5-digit strings
detention_facilities_clean['zip'] = (
    detention_facilities_clean['zip']
    .astype(int)
    .astype(str)
    .str.zfill(5)
)

# Current Guaranteed Minimum has too many missing values and we can use the
    ↪ total population column instead, so we will drop this column
detention_facilities_clean = detention_facilities_clean.
    ↪ drop(columns=['current_guaranteed_minimum'])

# State has 3 missing values, investigate:
print(detention_facilities_clean[detention_facilities_clean['state'].isna()])

# Impute state based on zip code for missing values
detention_facilities_clean.loc[detention_facilities_clean['state'].isna(),
    ↪ 'state'] = detention_facilities_clean.
    ↪ loc[detention_facilities_clean['state'].isna(), 'zip'].map({
        '92301': 'CA',
        '33073': 'FL',
        '71342': 'LA',
        '92301': 'CA'
    })

# Add year column by extracting from current_as_of column
detention_facilities_clean['year'] = pd.
    ↪ to_datetime(detention_facilities_clean['current_as_of']).dt.year
```

```
# Drop duplicates
detention_facilities_clean = detention_facilities_clean.drop_duplicates()

# Summarize
print(detention_facilities_clean.info())
```

	name	city	state	\
713	DESERT VIEW ANNEX	ADELANTO	NaN	
895	BROWARD TRANSITIONAL CENTER	POMPANO BEACH	NaN	
906	CENTRAL LOUISIANA ICE PROCESSING CENTER (CLIPC)	JENA	NaN	
931	DESERT VIEW ANNEX	ADELANTO	NaN	

	zip	type_detailed	average_daily_population	current_as_of
713	92301	CDF	385.0	2025-12-11
895	33073	CDF	674.0	2025-11-28
906	71342	IGSA	1135.0	2025-11-28
931	92301	CDF	400.0	2025-11-28

<class 'pandas.core.frame.DataFrame'>

Index: 17745 entries, 1 to 17890

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	name	17745 non-null	object
1	city	17745 non-null	object
2	state	17745 non-null	object
3	zip	17745 non-null	object
4	type_detailed	17745 non-null	object
5	average_daily_population	17745 non-null	float64
6	current_as_of	17745 non-null	datetime64[ns]
7	year	17745 non-null	int32

dtypes: datetime64[ns](1), float64(1), int32(1), object(5)

memory usage: 1.2+ MB

None

```
[57]: # Preview
detention_facilities_clean.head()
```

	name	city	state	zip	type_detailed	\
1	ADAMS COUNTY CORRECTIONAL CENTER	NATCHEZ	MS	39120	DIGSA	
2	ADELANTO ICE PROCESSING CENTER	ADELANTO	CA	92301	CDF	
3	ALEXANDRIA STAGING FACILI	ALEXANDRIA	LA	71303	STAGING	
4	ALLEGANY COUNTY JAIL	BELMONT	NY	14813	USMS IGA	
5	ALLEN PARISH PUBLIC SAFETY COMPLEX	OBERLIN	LA	70655	IGSA	

	average_daily_population	current_as_of	year
1	2191.0	2026-02-05	2026
2	1802.0	2026-02-05	2026
3	410.0	2026-02-05	2026

4	6.0	2026-02-05	2026
5	148.0	2026-02-05	2026

4 Panel Data Construction

Merge datasets across geography and time to create panel data for analysis using SPARK.

```
[63]: # Merge datasets on geographic identifiers (county/ZIP) and time
# Steps:
# 1. Standardize geographic keys across datasets
# 2. Align temporal granularity (monthly/yearly)
# 3. Join datasets using PySpark
# 4. Create balanced panel (if needed)

# Join places_zip_all_years with detention_facilities_clean on ZIP code and year
df_merged = pd.merge(
    places_zip_all_years,
    detention_facilities_clean,
    left_on=['zip', 'year'],
    right_on=['zip', 'year'],
    how='inner' # Use inner join to keep only matching records
)
```

5 Exploratory Data Analysis

Examine basic statistics, distributions, and patterns in each dataset.

```
[ ]: # Summary statistics and initial exploration
# - Descriptive statistics for each dataset
# - Time series trends
# - Geographic distribution
# - Correlation analysis

print("Exploratory data analysis code will go here")
```

6 Modeling

Analyze the relationship between immigration enforcement intensity and healthcare utilization.

6.1 Model Specification

Define control variables and modeling approach.

```
[ ]: # Modeling approach:
# - Panel regression with fixed effects
# - Control variables: socioeconomic status, demographics, healthcare,
  ↳ infrastructure
```

```

# - Dependent variable: Healthcare utilization (routine checkup rates)
# - Independent variable: Immigration enforcement intensity (detention counts)
# - Subgroup analysis by citizenship status and facility type

print("Model specification code will go here")

```

6.2 Model Estimation

Run regression models and generate results.

```

[ ]: # Convert to pandas for statsmodels
# panel_df = panel_data.toPandas()

# Fit regression model
# model = ols('healthcare_utilization ~ enforcement_intensity + controls',
#             ↪data=panel_df).fit()

# Display results
# print(model.summary())

print("Model estimation code will go here")

```

7 Data Visualization

Create maps, scatter plots, and regression visualizations to illustrate findings.

7.1 Geographic Distribution Maps (optional)

Create choropleth maps showing enforcement intensity and healthcare utilization across counties.

```

[ ]: # Create geographic visualizations
# - Map 1: Immigration enforcement intensity by county
# - Map 2: Healthcare utilization rates by county
# - Map 3: Combined visualization showing correlation

print("Geographic visualization code will go here")

```

7.2 Scatter Plots and Regression Lines

Visualize the relationship between enforcement intensity and healthcare utilization.

```

[ ]: # Create scatter plots with regression lines
# - Overall relationship
# - By subgroups (citizenship status, facility type)
# - Time period comparisons

print("Scatter plot visualization code will go here")

```

7.3 Time Series Analysis

Visualize trends in enforcement and healthcare utilization over time.

```
[ ]: # Create time series visualizations
# - Trends in ICE enforcement activity (2015-2025)
# - Trends in healthcare utilization rates
# - Key policy change periods (e.g., January 2025 sensitive location policy)

print("Time series visualization code will go here")
```

8 Results Summary

Summary of key findings and implications for the research article.

```
[ ]: # Key findings:
# 1. Geographic areas with high enforcement and low healthcare utilization
# 2. Statistical significance of the relationship
# 3. Subgroup differences
# 4. Policy implications

# Export results for inclusion in research article
print("Results summary and export code will go here")

# Note: This appendix supports the findings presented in:
# "Health Care Utilization and Immigration Enforcement" by Naomi Buell and ↵
↵Richie Rivera
```